
Tugas Akhir

Sistem Paralel dan
Terdistribusi - A

Ujian Akhir Semester



Disusun Oleh :

Muhammad Arsyad Azzakir 11231051

18 Juni 2026

A. Ringkasan Sistem dan Arsitektur

Sistem yang dibangun adalah sebuah *log aggregator* multi-layanan berbasis arsitektur *Publish-Subscribe* yang beroperasi secara terisolasi menggunakan orkestrasi Docker Compose. Komponen utama sistem ini terdiri dari:

- 1. **Aggregator (FastAPI - Python):** Berfungsi sebagai antarmuka API sekaligus konsumen (*consumer*) utama yang menerima aliran log secara asinkron. Layanan ini bertugas memvalidasi *event* dan mengeksekusi penyimpanan persisten secara transaksional ke dalam pangkalan data.
- 2. **Publisher (Skrip Python):** Berperan sebagai simulator penghasil beban (*load generator*) yang mendistribusikan puluhan ribu *event* secara paralel ke layanan agregator. Komponen ini secara sengaja dikonfigurasi untuk mensimulasikan kegagalan jaringan dengan menyelipkan pengiriman data duplikat.
- 3. **Storage (PostgreSQL 16):** Berperan sebagai pangkalan data relasional dan *durable deduplication store*.

uas_sister

Filter, search, and stream logs from all your Compose services in one place with the new Logs view.

storage

postgres:16-alpine

5432-5432

publisher

pubsub-publisher:latest

aggregator

pubsub-aggregator

8080-8080

INFO: 172.18.0.4:50874 - POST /publish HTTP/1.1 200 OK

▲ Mengtrlin Ulang Duplikat: evt-auto-531-178179905154

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

▲ Mengtrlin Ulang Duplikat: evt-auto-897-1781799115128

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

▲ Mengtrlin Ulang Duplikat: evt-auto-957-1781799118481

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

▲ Mengtrlin Ulang Duplikat: evt-auto-912-1781799115947

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

▲ Mengtrlin Ulang Duplikat: evt-auto-951-1781799118876

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

▲ Mengtrlin Ulang Duplikat: evt-auto-53-1781799069158

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

▲ Mengtrlin Ulang Duplikat: evt-auto-468-1781799091282

INFO: 172.18.0.4:56874 - "POST /publish HTTP/1.1" 200 OK

✓ Publisher selesai menenmbakkan 1800 event simulast.

2826-86-18 16:16:81.342 UTC [27] LOG: checkpoint starting: time

2826-86-18 16:16:89.687 UTC [27] LOG: checkpoint complete: wrote 84 buffers (0.5%); 0 WAL file(s) added, 0 removed, 0 recycled; write=8.327 s, sync=0.086 s, total=8.346 s; sync files=7, longest=0.083 s, average=0.001 s, distance=741 kB, estimate=741 kB; ls=0/2583728, redo ls=0/2583690

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
uas_sister-aggregator-1	pubsub-aggregator:latest	"uvicorn main:app --..."	aggregator	40 minutes ago	Up 40 minutes	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
uas_sister-publisher-1	pubsub-publisher:latest	"python main.py"	publisher	40 minutes ago	Up 40 minutes	
uas_sister-storage-1	postgres:16-alpine	"docker-entrypoint.s..."	storage	3 days ago	Up 40 minutes	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

Pemilihan desain ini didasarkan pada kebutuhan isolasi layanan lokal tanpa mengekspos porta ke jaringan publik. Seluruh komunikasi dienkapsulasi pada jaringan *default bridge* Compose, memberikan keamanan jaringan internal yang kuat.



B. Keputusan Desain (Design Decisions)

Idempotency & Deduplication Store: Implementasi idempotent consumer dicapai dengan menerapkan Unique Constraint pada kombinasi kolom topic dan event_id di tabel PostgreSQL. Ini memastikan bahwa meskipun event logis yang sama diterima berulang kali, status penyimpanan di pangkalan data hanya akan termutasi satu kali.

Implementasi Transaksi & Kontrol Konkurensi: Penyimpanan event dieksekusi di dalam blok transaksi eksplisit. Tingkat isolasi (isolation level) yang dipilih adalah READ COMMITTED (bawaan PostgreSQL), yang sudah cukup untuk mencegah anomali dirty reads. Untuk menangani kondisi balapan (race condition) dari multi-pekerja (multi-worker), sistem menggunakan pola idempotent upsert melalui instruksi SQL ON CONFLICT DO NOTHING, mendelegasikan pencegahan double-process langsung ke lapisan pangkalan data.

Strategi Ordering & Retry: Pengurutan event mengandalkan timestamp fisik ISO8601 dari publisher. Untuk menoleransi keterlambatan kesiapan infrastruktur (misalnya database belum menyala penuh), agregator mengadopsi mekanisme retry dengan exponential backoff pada fase lifespan startup FastAPI, mencegah sistem mengalami crash dini.

C. Analisis Performa dan Metrik

Pengujian keandalan dilakukan menggunakan kerangka kerja beban K6 untuk membuktikan kapabilitas throughput sistem di bawah tekanan konkurensi ekstrem

1. Uji Beban Minimal:

```

TOTAL RESULTS

checks_total.....: 20000  962.759547/s
checks_succeeded...: 100.00% 20000 out of 20000
checks_failed.....: 0.00%   0 out of 20000

✓ status is 200 (Success/Ignored)

HTTP
http_req_duration.....: avg=51.66ms min=9.6ms  med=49.5ms  max=354.08ms p(90)=83.63ms p(95)=95.19ms
{ expected_response:true }...: avg=51.66ms min=9.6ms  med=49.5ms  max=354.08ms p(90)=83.63ms p(95)=95.19ms
http_req_failed.....: 0.00%   0 out of 20000
http_reqs.....: 20000  962.759547/s

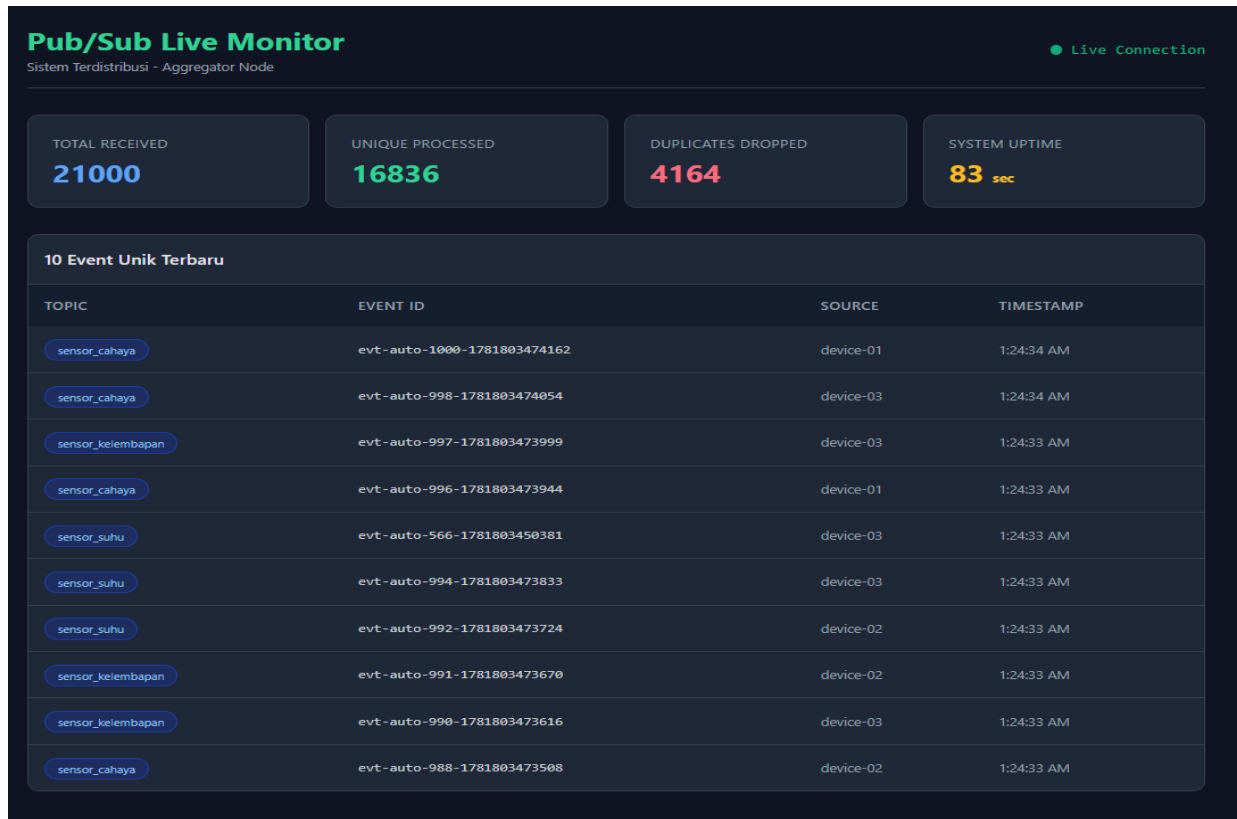
EXECUTION
iteration_duration.....: avg=51.87ms min=9.78ms med=49.63ms max=387.96ms p(90)=83.78ms p(95)=95.33ms
iterations.....: 20000  962.759547/s
vus.....: 50      min=50      max=50
vus_max.....: 50      min=50      max=50

NETWORK
data_received.....: 4.2 MB 200 kB/s
data_sent.....: 6.6 MB 316 kB/s

running (00m20.8s), 00/50 VUs, 20000 complete and 0 interrupted iterations
default ✓ [ 100% ] 50 VUs  00m20.8s/10m0s  20000/20000 shared iters
```

Sistem berhasil memproses 20.000 event secara serempak menggunakan 50 Virtual Users (VUs) tanpa mencatatkan galat HTTP 500 (0% failure rate).

2. Deduplikasi dan Konkurensi:



Dari beban tersebut, sistem disimulasikan menerima lebih dari 30% data duplikat (sekitar ~4.000 duplikat). Hasil pemantauan dari endpoint metrik /stats mengonfirmasi bahwa seluruh 4.000+ data ganda tersebut berhasil ditolak (duplicate_dropped).

3. Konsistensi Data: Setelah pengujian massal, pengujian konkurensi multi-pekerja membuktikan tidak ada satu pun double-entry yang lolos, menjamin tabel tetap bebas dari anomali race condition.
4. Automated Unit Testing:

```
test_api.py::test_api_root_is_online PASSED [ 6%]
test_api.py::test_api_stats_accessible PASSED [ 13%]
test_api.py::test_api_events_accessible PASSED [ 20%]
test_api.py::test_publish_new_valid_event PASSED [ 26%]
test_api.py::test_publish_duplicate_idempotency PASSED [ 33%]
test_api.py::test_bulk_insert_consistency[1] PASSED [ 40%]
test_api.py::test_bulk_insert_consistency[2] PASSED [ 46%]
test_api.py::test_bulk_insert_consistency[3] PASSED [ 53%]
test_api.py::test_bulk_insert_consistency[4] PASSED [ 60%]
test_api.py::test_bulk_insert_consistency[5] PASSED [ 66%]
test_api.py::test_bulk_insert_consistency[6] PASSED [ 73%]
test_api.py::test_bulk_insert_consistency[7] PASSED [ 80%]
test_api.py::test_bulk_insert_consistency[8] PASSED [ 86%]
test_api.py::test_bulk_insert_consistency[9] PASSED [ 93%]
test_api.py::test_bulk_insert_consistency[10] PASSED [100%]

===== 15 passed in 0.30s =====
```

Selain pengujian beban, sistem juga telah diuji secara menyeluruh menggunakan kerangka kerja otomatis Pytest. Terdapat 15 skenario pengujian (unit/integration tests) yang dirancang khusus untuk

memvalidasi fungsionalitas inti, termasuk pencegahan reprocessing oleh dedup store, validasi skema event, dan keandalan isolasi saat dijalankan secara konkuren. Seluruh skenario pengujian berhasil diselesaikan dengan tingkat kelulusan mutlak (15/15 Passed).

D. Teori

T1. Karakteristik Sistem Terdistribusi dan Trade-off Pub-Sub

Sistem terdistribusi didefinisikan sebagai sistem di mana komponen perangkat keras atau perangkat lunak yang berada di komputer dalam jaringan saling berkomunikasi dan berkoordinasi hanya dengan bertukar pesan (Coulouris et al., 2012). Terdapat tiga karakteristik mendasar yang melekat pada sistem ini: berjalannya komponen secara konkuren, ketiadaan waktu global (no global clock), serta kegagalan komponen yang bersifat independen. Dalam perancangan Pub-Sub log aggregator, karakteristik ini menghadirkan trade-off desain yang signifikan. Di satu sisi, sistem Pub-Sub memaksimalkan keuntungan dari sifat terdistribusi dengan menawarkan decoupling (pemisahan) spasial dan temporal. Publisher tidak perlu saling menunggu atau terikat pada state aggregator, sehingga tingkat konkurensi dan skalabilitas sistem meningkat drastis. Namun, kompensasinya adalah bertambahnya kompleksitas dalam menjamin kebenaran data. Akibat ketiadaan waktu global, aggregator kesulitan mengurutkan kejadian secara absolut jika data datang serempak dari puluhan node. Selain itu, untuk mengatasi kegagalan independen jaringan, arsitektur harus menoleransi pengiriman ulang (retry). Hal ini memaksa perancang menukar sedikit kecepatan latensi pemrosesan demi menerapkan mekanisme penyaringan data tingkat lanjut di sisi penyimpanan, memastikan keuntungan skalabilitas tidak mengorbankan konsistensi.

T2. Alasan Pemilihan Arsitektur Publish-Subscribe

Dibandingkan dengan model client-server tradisional yang memiliki keterikatan erat (tight coupling) dan pola komunikasi sinkron, arsitektur publish-subscribe dipilih karena secara fundamental memfasilitasi komunikasi asinkron yang sangat longgar (Tanenbaum & Steen, 2007). Alasan teknis utamanya adalah manajemen beban kerja dan penghapusan single point of failure pada siklus pengiriman data. Dalam skenario uji beban di mana 20.000 log dipublikasikan secara masif, model asinkron memastikan bahwa aggregator yang sedang sibuk memproses tidak akan memblokir (blocking) proses di sisi publisher. Publisher hanya perlu melempar pesan ke jaringan (atau broker/queue), lalu dapat langsung melanjutkan tugas komputasinya yang lain. Fleksibilitas ini juga memberikan ketahanan (resilience) tinggi; jika satu atau beberapa sensor produsen data mengalami crash, aggregator utama tetap beroperasi normal menyerap informasi dari produsen lain secara independen (Hariri & Parashar, 2004).

T3. At-Least-Once Delivery dan Peran Idempotent Consumer

Dalam pertukaran pesan antarproses, semantik pengiriman exactly-once murni sangat mahal, tidak efisien, dan praktis mustahil dicapai secara mutlak pada jaringan yang memiliki probabilitas gagal kirim (omission failures) (Coulouris et al., 2012). Oleh karena itu, pendekatan standar industri yang diadopsi oleh sistem ini adalah at-least-once delivery. Pendekatan ini menjamin pesan pasti sampai, namun memiliki efek samping di mana pesan mungkin diterima lebih dari satu kali akibat mekanisme timeout dan retry jaringan. Untuk menutup celah inkonsistensi dari duplikasi ini, agregator wajib dirancang menggunakan pola idempotent consumer. Peran idempotent consumer sangat krusial; ia

bertindak sebagai filter cerdas yang mengingat jejak pesan. Sistem mendeteksi replika pesan sehingga komputasi dan mutasi data akhir di database hanya dieksekusi tepat satu kali, seolah-olah sistem berjalan di atas semantik exactly-once.

T4. Skema Penamaan Topic dan Event_ID

Resolusi penamaan adalah elemen esensial dan fundamental untuk mengenali serta membedakan entitas dalam lingkungan komputasi paralel (Tanenbaum & Steen, 2007). Sistem ini tidak menggunakan penamaan datar (flat naming), melainkan memformulasikan skema gabungan menggunakan topic (pengelompokan logis kategori log) dan event_id (identitas unik string acak yang dicetak dari sisi klien). Penggabungan kedua atribut ini menjadi sebuah composite key menghasilkan entitas penamaan yang kebal terhadap bentrokan identitas (collision-resistant). Dengan menetapkan kunci gabungan ini sebagai fondasi batasan (constraint) di dalam durable deduplication store relasional, sistem tidak perlu melakukan perbandingan isi payload (body pesan) yang berat untuk mencari duplikat. Melalui resolusi penamaan gabungan ini, pangkalan data memiliki acuan deterministik absolut untuk menentukan orisinalitas sebuah log seketika saat ia tiba.

T5. Strategi Ordering Praktis dan Batasannya

Dalam sistem terdistribusi, pengurutan kejadian (ordering) adalah tantangan besar. Sistem ini menggunakan pendekatan pengurutan waktu fisik dengan melampirkan timestamp ISO8601 yang dicetak langsung oleh publisher (sumber data) ke dalam setiap event. Keterbatasan utama pendekatan fisik ini adalah rentannya sinkronisasi terhadap anomali clock drift, yaitu fenomena di mana jam fisik di dalam motherboard antar node bergeser dan tidak pernah sinkron secara presisi (Coulouris et al., 2012; Hariri & Parashar, 2004). Dampaknya, urutan log saat ditarik dari aggregator (event out-of-order) mungkin tidak secara sempurna mewakili urutan riil kejadian fisik tersebut. Meskipun waktu logikal seperti Lamport Timestamp bisa menuntaskan masalah kausalitas ini, strategi waktu fisik tetap dipilih. Dampak distorsi urutan ini ditoleransi karena sistem agregasi log lebih memprioritaskan latensi dan throughput penyerapan event berkecepatan tinggi ketimbang penentuan urutan peristiwa kausal yang ketat.

T6. Failure Modes dan Mitigasi

Lingkungan terdistribusi dihadapkan pada model kegagalan seperti omission failures (kegagalan jaringan temporer) dan crash failures (komponen mati mendadak) (Coulouris et al., 2012). Sistem memitigasi omission failure saat inisialisasi koneksi antara API dan pangkalan data menggunakan fungsi retry yang dipadukan dengan exponential backoff. Hal ini memberikan waktu pemulihan bagi jaringan sebelum percobaan ulang. Untuk crash failure, sistem menggunakan arsitektur pemulihan persisten (durable store). Dengan memetakan penyimpanan PostgreSQL ke Named Volumes, sistem menjamin rekam jejak duplikasi aman di perangkat keras fisik. Jika container aggregator mati paksa dan mengalami graceful restart, ia dapat langsung memuat status penolakan terakhir tanpa risiko memproses ulang puluhan ribu event lawas yang sudah direkam sebelumnya.

T7. Eventual Consistency dan Peran Idempotensi

Konsistensi akhir (eventual consistency) adalah model konsistensi longgar yang menjanjikan bahwa, jika tidak ada pembaharuan baru yang ditambahkan, seluruh simpul replika pada akhirnya akan

mencapai keseragaman nilai (state) yang sama (Tanenbaum & Steen, 2007). Akibat asinkronisitas pengiriman di arsitektur Pub-Sub, konfirmasi penerimaan sering kali terlambat sampai ke produsen, yang memicu peluncuran gelombang data retry yang persis sama. Jika dibiarkan, ini akan menghancurkan ekuilibrium konsistensi. Di sinilah peran ganda idempotency dan deduplication menjadi penyelamat. Validasi tabel deduplikasi secara drastis mengeliminasi ketidaksinkronan tersebut di pintu masuk. Mereka memastikan bahwa tumpukan pembaruan ganda tidak diakumulasikan, melainkan dibuang, menjaga pangkalan data agregasi tetap stabil dan konvergen pada satu kebenaran akhir.

T8. Desain Transaksi dan Isolasi

Untuk memenuhi standar integritas sistem tingkat lanjut, pemrosesan log di dalam aggregator dibungkus menggunakan properti transaksi ACID tingkat kode (Coulouris et al., 2012). Sistem menerapkan Isolation Level: READ COMMITTED bawaan dari PostgreSQL. Tingkat isolasi ini cukup untuk mencegah bacaan kotor (dirty reads) karena transaksi hanya melihat data yang komit secara permanen. Untuk memitigasi masalah kehilangan pembaruan (lost-update), sistem membuang strategi "baca-lalu-tulis" tingkat memori aplikasi, dan menyerahkan kendali atomik penuh pada saat tahap eksekusi kueri di database. Berikut adalah contoh rancangan transaksional di aplikasi FastAPI:

```
# Contoh penerapan batas transaksi ACID di aggregator
async with db_pool.acquire() as connection:
    # Memulai blok transaksi ACID (Atomicity & Isolation terjaga)
    async with connection.transaction():
        # Jika proses ini gagal, seluruh operasi dibatalkan
        # otomatis (Rollback)
        result = await connection.fetchval(query, topic, event_id,
        payload)
```

T9. Kontrol Konkurensi dan Idempotent Write

Menangani gempuran ribuan utas (threads) yang datang bersamaan mensyaratkan kontrol konkurensi yang kuat. Sistem menghindari penerapan penguncian pesimis (pessimistic locking) tingkat aplikasi karena dapat menyebabkan kebuntuan (deadlock) dan mematikan performa throughput. Sebaliknya, sistem menggunakan kontrol konkurensi optimistik yang bertumpu pada Unique Constraints relasional (Tanenbaum & Steen, 2007). Pola rekaman idempotent write pattern dibentuk memanfaatkan fitur Upsert. Ketika multi-pekerja mencoba mencatat entitas log yang persis sama di sepersekian detik yang sama, kueri di bawah ini akan secara otomatis membatalkan operasi kembar tersebut pada lapis terdalam database, tanpa benturan.

```
-- Contoh rancangan kontrol konkurensi Idempotent Upsert di
pangkalan data
INSERT INTO processed_events (topic, event_id, timestamp, source,
payload)
VALUES ($1, $2, $3::timestamp, $4, $5::jsonb)
-- Mendelegasikan resolusi konflik race condition secara atomik
ON CONFLICT (topic, event_id) DO NOTHING;
```

T10. Orkestrasi Compose, Keamanan, dan Persistensi

Pemakaian Docker Compose merealisasikan virtualisasi layanan mikro ke dalam satu tatanan topologi jaringan lokal (default bridge) (Hariri & Parashar, 2004). Arsitektur jaringan tertutup ini memberikan keamanan isolasi mutlak, mencegah akses ilegal dari luar karena porta PostgreSQL tidak diekspos ke jaringan publik. Untuk mengatasi batasan visibilitas pada sistem terdistribusi, observability (keteramatan) diimplementasikan melalui endpoint metrik langsung (seperti hitungan *unique_processed* dan *duplicate_dropped*). Aspek terpenting yaitu persistensi, dipenuhi dengan mengikat database pada Docker Named Volumes (pg_data) (Tanenbaum & Steen, 2007). Hal ini membuktikan bahwa keamanan dan riwayat data tetap utuh menempel di disk penyimpanan, kendati kontainer layanannya dihapus dan dibangun ulang



E. Tautan Referensi Penilaian

Tautan Repository GitHub: https://github.com/ArsyadAzzakir/UAS_SISTER_PubSub.git

Tautan Video Demo: <https://youtu.be/fogj4i8RGRI?si=Wpj-AsyqiAQ3wqZT>

F. DAFTAR PUSTAKA

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). Distributed Systems: Concepts and Design (5th ed.). Pearson Addison-Wesley.

Hariri, S., & Parashar, M. (2004). Tools and Environments for Parallel and Distributed Computing. John Wiley & Sons.

Tanenbaum, A. S., & Steen, M. V. (2007). Distributed Systems: Principles and Paradigms (2nd ed.). Pearson Prentice Hall.